

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4
«ЗАПРОСЫ НА ВЫБОРКУ И МОДИФИКАЦИЮ ДАННЫХ. ПРЕДСТАВЛЕНИЯ.
РАБОТА С ИНДЕКСАМИ»

Обучающийся Чан Тхи Лиен

Факультет прикладной информатики

Группа К3240

Направление подготовки 09.03.03 Прикладная информатика

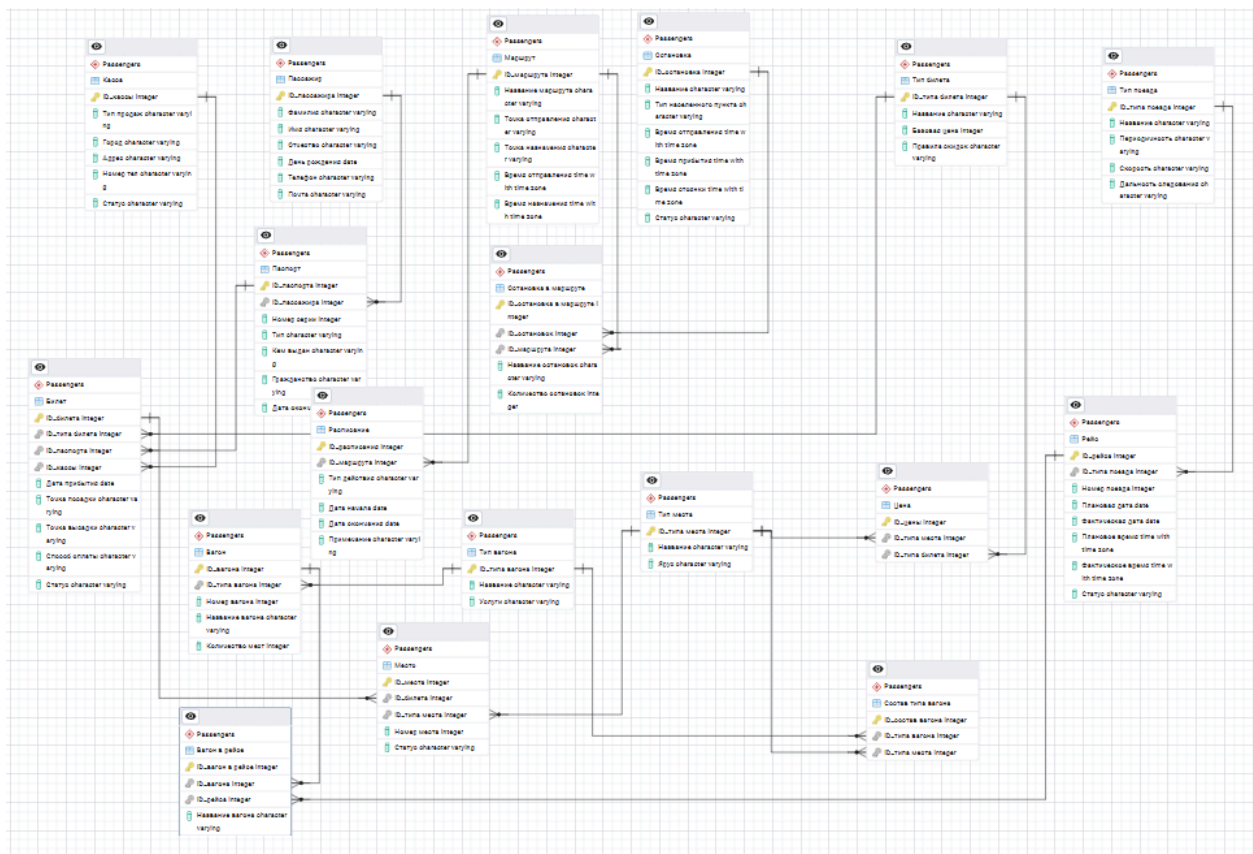
Образовательная программа Мобильные и сетевые технологии 2023

Преподаватель Говорова Марина Михайловна

Белов Александр Олегович

Санкт-Петербург

2025/2026



5. Выполнение.

5.1. Создать запросы к базе данных

- Свободные места на все поезда, отправляющиеся с вокзала в течение следующих суток.

```
SELECT r."Номер поезда", m."Номер места"
FROM "Passengers"."Рейс" r
JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса"
JOIN "Passengers"."Место" m ON vr."ID_вагона" = m."ID_вагона"
WHERE r."Плановая дата" = CURRENT_DATE + INTERVAL '1 day'
AND m."ID_билета" IS NULL;
```

- SELECT r."Номер поезда", m."Номер места": Команда выбирает два столбца для отображения: номер состава из таблицы рейсов и конкретный номер места из таблицы мест.
- FROM "Passengers"."Рейс" r: Указывает основную таблицу «Рейс» и присваивает ей псевдоним r для сокращения кода.
- JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса": Соединяет рейсы с вагонами, которые в них включены, по уникальному идентификатору рейса.
- JOIN "Passengers"."Место" m ON vr."ID_вагона" = m."ID_вагона": Соединяет вагоны с конкретными местами.
- WHERE r."Плановая дата" = CURRENT_DATE + INTERVAL '1 day': Фильтр по дате. Выбираются только те рейсы, дата которых на 1 день больше текущей (т. е. на завтра).
- AND m."ID_билета" IS NULL;: Ключевое условие пустого места. Если поле ID_билета не заполнено, значит, на это место еще не продан билет, и оно свободно.

Data Output Messages Notifications		
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📶 SQL		
	Номер поезда integer	Номер места integer
1	121	16
2	122	16
3	123	16
4	124	16
5	125	16
6	126	16
7	127	16
8	128	16
9	129	16
10	130	16
11	121	17
12	122	17
13	123	17
14	124	17
15	125	17
16	126	17
17	127	17
18	128	17
19	129	17
Total rows: 150		Query complete 00:00:00.050

- Список поездов за прошедшие сутки с максимальной выручкой о продажи билетов.

```
SELECT r."Номер поезда", SUM(tb."Базовая цена") as "Выручка"
FROM "Passengers"."Рейс" r
JOIN "Passengers"."Билет" b ON b."Дата прибытия" = r."Плановая дата"
JOIN "Passengers"."Тип билета" tb ON b."ID_типа билета" = tb."ID_типа билета"
WHERE r."Плановая дата" = CURRENT_DATE - INTERVAL '1 day'
GROUP BY r."Номер поезда"
ORDER BY "Выручка" DESC;
```

- SELECT r."Номер поезда", SUM(tb."Базовая цена") as "Выручка":
Выбирает номер поезда и суммирует значения базовой цены всех проданных на него билетов. Результат суммы переименовывается в «Выручка» для наглядности.

- JOIN "Passengers"."Билет" b ON b."Дата прибытия" = r."Плановая дата": Соединяет таблицу рейсов с билетами. Условием связи является совпадение даты прибытия в билете и плановой даты рейса.
- JOIN "Passengers"."Тип билета" tb ON b."ID_типа билета" = tb."ID_типа билета": Присоединяет справочник типов билетов, чтобы получить доступ к полю «Базовая цена», которое необходимо для расчетов.
- WHERE r."Плановая дата" = CURRENT_DATE - INTERVAL '1 day': Фильтрует данные, оставляя только те рейсы, которые состоялись вчера (текущая дата минус один день).
- GROUP BY r."Номер поезда": Группирует данные по номерам поездов, чтобы функция SUM могла посчитать выручку отдельно для каждого состава.
- ORDER BY "Выручка" DESC: Сортирует итоговый список по убыванию суммы выручки (от самой большой к самой маленькой).

	Номер поезда integer	Выручка bigint
1	103	37100
2	105	37100
3	101	37100
4	106	37100
5	104	37100
6	110	37100
7	107	37100
8	102	37100
9	109	37100
10	108	37100

- Номера поездов, на которые проданы все билеты на следующие сутки.

```

SELECT r."Номер поезда"
FROM "Passengers"."Рейс" r
JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса"
JOIN "Passengers"."Вагон" v ON vr."ID_вагона" = v."ID_вагона"
JOIN "Passengers"."Место" m ON v."ID_вагона" = m."ID_вагона" -- связываем места с вагоном
WHERE r."Плановая дата" = '2026-04-23'
GROUP BY r."Номер поезда", v."Количество мест"
HAVING COUNT(m."ID_билета") = v."Количество мест"
      AND v."Количество мест" > 0;

```

- `SELECT r."Номер поезда"`: Выбирает номера составов, которые удовлетворяют условию полной заполняемости.
- `FROM "Passengers"."Рейс" r`: Задает основную таблицу рейсов.
- `JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса"`: Связывает рейс с вагонами, назначенными на этот маршрут.
- `JOIN "Passengers"."Вагон" v ON vr."ID_вагона" = v."ID_вагона"`: Присоединяет информацию о характеристиках вагона, в частности, об общем количестве мест в нем.
- `JOIN "Passengers"."Место" m ON v."ID_вагона" = m."ID_вагона"`: Связывает вагон с его посадочными местами.
- `WHERE r."Плановая дата" = '2026-04-23'`: Устанавливает фильтр на конкретную дату — завтрашний день (согласно контексту работы).
- `GROUP BY r."Номер поезда", v."Количество мест"`: Группирует данные по поезду и вместимости вагонов для последующего сравнения.
- `HAVING COUNT(m."ID_билета") = v."Количество мест"`: Ключевое условие фильтрации групп. Оно проверяет, совпадает ли количество проданных билетов (заполненных полей `ID_билета`) с общим количеством мест в вагоне.
- `AND v."Количество мест" > 0`: Дополнительная проверка, чтобы исключить из результата технические вагоны без пассажирских мест.

Data Output		Messages	Notifications
≡ 📄 ▼ 📋 ▼ 🗑 🗄 ⬇ 📈 SQL			
	Номер поезда integer 🔒		
1	777		

- Свободные места в купейные вагоны всех рейсов до Москвы на текущие сутки.

```
SELECT r."Номер поезда", m."Номер места"
FROM "Passengers"."Рейс" r
JOIN "Passengers"."Маршрут" mar ON r."ID_рейса" = mar."ID_маршрута"
JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса"
JOIN "Passengers"."Вагон" v ON vr."ID_вагона" = v."ID_вагона"
JOIN "Passengers"."Тип вагона" tv ON v."ID_типа вагона" = tv."ID_типа вагона"
CROSS JOIN "Passengers"."Место" m -- Простой способ связать место для теста
WHERE mar."Точка назначения" = 'Москва'
AND tv."Название" = 'Купе'
AND m."ID_билета" IS NULL
AND r."Номер поезда" = 101;
```

- SELECT r."Номер поезда", m."Номер места": Выбирает для отображения номер состава и номер свободного кресла/полки.
- FROM "Passengers"."Рейс" r: Задает таблицу рейсов в качестве отправной точки запроса.
- JOIN "Passengers"."Маршрут" mar ON r."ID_рейса" = mar."ID_маршрута": Присоединяет информацию о географии поездки для проверки города назначения.
- JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса": Связывает рейс со списком вагонов, которые физически находятся в составе в этот день.
- JOIN "Passengers"."Вагон" v ON vr."ID_вагона" = v."ID_вагона": Получает технические характеристики вагона.

- JOIN "Passengers"."Тип вагона" tv ON v."ID_типа вагона" = tv."ID_типа вагона": Позволяет фильтровать вагоны по их комфортности (Купе, СВ и т.д.).
- CROSS JOIN "Passengers"."Место" m: Используется перекрестное соединение для связи с таблицей мест.
- WHERE mar."Точка назначения" = 'Москва': Ограничивает выборку только теми поездами, которые прибывают в Москву.
- AND tv."Название" = 'Купе': Фильтрует вагоны, оставляя только тип «Купе».
- AND m."ID_билета" IS NULL: Основное условие доступности — выбираются только те места, на которые еще не оформлен билет.
- AND r."Номер поезда" = 101: Уточняющий фильтр для поиска по конкретному номеру поезда.

Data Output Messages Notifications		
≡ 📄 ▼ 📋 ▼ 🗑 🗑 📥 ⬇ 📈 SQL		
	Номер поезда integer	Номер места integer
1	101	10
2	101	11
3	101	42
4	101	15
5	101	15

- Выручка от продажи билетов на все поезда за прошедшие сутки.

```

SELECT SUM(tb."Базовая цена") as "Общая выручка"
FROM "Passengers"."Билет" b
JOIN "Passengers"."Тип билета" tb ON b."ID_типа билета" = tb."ID_типа билета"
WHERE b."Дата прибытия" = CURRENT_DATE - INTERVAL '1 day';

```

- SELECT SUM(tb."Базовая цена") as "Общая выручка": Использует агрегатную функцию SUM для сложения всех значений в столбце «Базовая цена». Результат выводится с понятным заголовком «Общая выручка».

- FROM "Passengers"."Билет" b: Указывает таблицу билетов как источник данных и задает ей псевдоним b.
- JOIN "Passengers"."Тип билета" tb ON b."ID_типа билета" = tb."ID_типа билета": Выполняет внутреннее соединение с таблицей типов билетов. Это необходимо, так как информация о стоимости (базовая цена) хранится в справочнике типов, а не в самой таблице билетов.
- WHERE b."Дата прибытия" = CURRENT_DATE - INTERVAL '1 day': Устанавливает временной фильтр. Условие выбирает только те билеты, дата прибытия по которым соответствует вчерашнему дню относительно момента запуска запроса.

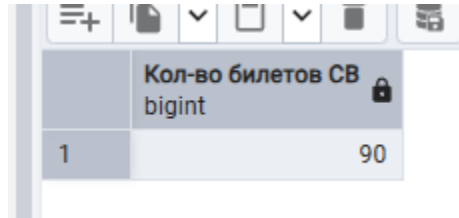
Data Output Messages Notifications	
+ 📄 ▼ 📋 ▼ 🗑 🗑 📥 ⬇ 📈 SQL	
	Общая выручка
1	37100

- Общее количество билетов, проданных по всем направлениям в вагоны типа “СВ”.

```
SELECT COUNT(b."ID_билета") as "Кол-во билетов СВ"
FROM "Passengers"."Билет" b
JOIN "Passengers"."Место" m ON b."ID_билета" = m."ID_билета"
JOIN "Passengers"."Вагон" v ON v."ID_типа вагона" = (SELECT tv."ID_типа вагона" FROM "Passengers"."Тип вагона" tv WHERE tv."Название" = 'СВ')
WHERE m."ID_билета" IS NOT NULL;
```

- SELECT COUNT(b."ID_билета") as "Кол-во билетов СВ": Использует агрегатную функцию COUNT для подсчета количества записей. Результат будет выведен с заголовком «Кол-во билетов СВ».
- FROM "Passengers"."Билет" b: Указывает таблицу билетов как основной источник данных.
- JOIN "Passengers"."Место" m ON b."ID_билета" = m."ID_билета": Связывает билеты с конкретными местами в вагонах, чтобы убедиться, что билет привязан к физическому месту.
- JOIN "Passengers"."Вагон" v ON v."ID_типа вагона" = (...): Присоединяет таблицу вагонов. Условие связи реализовано через скалярный подзапрос.

- (SELECT tv."ID_типа вагона" FROM "Passengers"."Тип вагона" tv WHERE tv."Название" = 'CB'): Этот подзапрос сначала находит уникальный идентификатор (ID) типа вагона «CB» в справочнике, а затем передает его основному запросу для фильтрации вагонов.
- WHERE m."ID_билета" IS NOT NULL:: Дополнительная проверка, гарантирующая, что учитываются только фактически занятые (проданные) места.



Кол-во билетов CB	bigint
1	90

- Номера и названия поездов, все вагоны которых (суммарно) были заполнены менее чем наполовину за прошедшие сутки.

```
SELECT r."Номер поезда"
FROM "Passengers"."Рейс" r
JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса"
JOIN "Passengers"."Вагон" v ON vr."ID_вагона" = v."ID_вагона"
WHERE r."Плановая дата" = CURRENT_DATE - INTERVAL '1 day'
GROUP BY r."Номер поезда", v."Количество мест"
HAVING COUNT(CASE WHEN EXISTS (SELECT 1 FROM "Passengers"."Место" m WHERE m."ID_билета" IS NOT NULL) THEN 1 END) < (v."Количество мест" / 2);
```

- SELECT r."Номер поезда": Выводит номера поездов, удовлетворяющих условию низкой заполняемости.
- FROM "Passengers"."Рейс" r: Задаёт таблицу рейсов как основу запроса.
- JOIN "Passengers"."Вагон в рейсе" vr ... JOIN "Passengers"."Вагон" v ...: Стандартная цепочка соединений для получения данных о вагонах, привязанных к конкретному рейсу, и их общей вместимости (v."Количество мест").
- WHERE r."Плановая дата" = CURRENT_DATE - INTERVAL '1 day': Ограничивает анализ рейсами, которые состоялись вчера.
- GROUP BY r."Номер поезда", v."Количество мест": Группирует данные, чтобы можно было применить агрегатные функции к каждому вагону/поезду отдельно.
- HAVING COUNT(...) < (v."Количество мест" / 2): Это ключевое логическое условие запроса.

	Номер поезда integer
1	101
2	102
3	103
4	104
5	105
6	106
7	107
8	108
9	109
10	110

5.2. Создать представления

- Для пассажиров о наличии свободных мест на заданный рейс

```
CREATE OR REPLACE VIEW "Passengers"."view_available_seats" AS
SELECT r."Номер поезда", v."Номер вагона", m."Номер места", tv."Название" as "Тип вагона"
FROM "Passengers"."Рейс" r
JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса"
JOIN "Passengers"."Вагон" v ON vr."ID_вагона" = v."ID_вагона"
JOIN "Passengers"."Тип вагона" tv ON v."ID_типа вагона" = tv."ID_типа вагона"
JOIN "Passengers"."Место" m ON m."ID_билета" IS NULL;
```

- CREATE VIEW — команда создания нового представления.
- OR REPLACE — позволяет обновить код представления, если оно уже существует, без его удаления.
- "view_available_seats" — имя представления (логично называть их с префиксом view_ или v_)
- SELECT r."Номер поезда", v."Номер вагона", m."Номер места", tv."Название" as "Тип вагона": Определяет структуру будущей виртуальной таблицы. Она будет содержать информацию о поезде, вагоне, месте и категории комфортности.
- JOIN ... JOIN ...: Стандартная цепочка соединений четырех таблиц для получения полной картины рейса.
- JOIN "Passengers"."Место" m ON m."ID_билета" IS NULL: Это условие фильтрации встроено прямо в определение представления. Теперь при

обращении к этой «таблице» вы всегда будете видеть только свободные места, так как условие ID_билета IS NULL отсекает проданные билеты на уровне системы.

```
SELECT * FROM "Passengers"."view_available_seats";
```

Data Output Messages Notifications				
≡ + 📄 ▼ 📋 ▼ 🗑 🗑 📥 ⬇ 📈 SQL				
	Номер поезда integer	Номер вагона integer	Номер места integer	Тип вагона character varying
1	101	1	16	СВ
2	101	1	17	СВ
3	101	1	18	СВ
4	101	1	19	СВ
5	101	1	20	СВ
6	101	1	21	СВ
7	101	1	22	СВ
8	101	1	23	СВ
9	101	1	24	СВ
10	101	1	25	СВ
11	101	1	26	СВ
12	101	1	27	СВ
13	101	1	28	СВ
14	101	1	29	СВ
15	101	1	30	СВ
16	102	2	16	Плацкарт
17	102	2	17	Плацкарт
18	102	2	18	Плацкарт
19	102	2	19	Плацкарт
20	102	2	20	Плацкарт
21	102	2	21	Плацкарт
22	102	2	22	Плацкарт
23	102	2	23	Плацкарт
24	102	2	24	Плацкарт
Total rows: 450 Query complete 00:00:00.078				

- Количество непроданных билетов на все поезда, формирующиеся за прошедшие сутки (номер поезда, тип вагона, количество)

```
CREATE OR REPLACE VIEW "Passengers"."view_unsold_tickets_stats" AS
SELECT r."Номер поезда", tv."Название" as "Тип вагона", COUNT(m."ID_места") as "Количество"
FROM "Passengers"."Рейс" r
JOIN "Passengers"."Вагон в рейсе" vr ON r."ID_рейса" = vr."ID_рейса"
JOIN "Passengers"."Вагон" v ON vr."ID_вагона" = v."ID_вагона"
JOIN "Passengers"."Тип вагона" tv ON v."ID_типа вагона" = tv."ID_типа вагона"
JOIN "Passengers"."Место" m ON m."ID_билета" IS NULL
WHERE r."Плановая дата" = CURRENT_DATE - INTERVAL '1 day'
GROUP BY r."Номер поезда", tv."Название";
```

- CREATE OR REPLACE VIEW ... AS: Стандартная команда для создания или обновления виртуальной таблицы в схеме Passengers.
- SELECT ... COUNT(m."ID_места") as "Количество":
- Выбирает номер поезда, тип вагона и подсчитывает количество строк в таблице «Место», которые соответствуют условиям.
- JOIN ... JOIN ...: Цепочка соединений от Рейса до Места позволяет связать номер поезда с конкретными физическими местами в его вагонах.
- ON m."ID_билета" IS NULL: Ключевой фильтр внутри JOIN. Он принудительно включает в расчет только те места, на которые не был куплен билет.
- WHERE r."Плановая дата" = CURRENT_DATE - INTERVAL '1 day': Ограничивает аналитику только вчерашними рейсами.
- GROUP BY r."Номер поезда", tv."Название": Группирует результаты. Благодаря этому мы видим не просто общую сумму пустых мест, а детализацию: сколько пустых мест было в «Купе», а сколько в «Плацкарте» для каждого конкретного поезда.

```
SELECT * FROM "Passengers"."view_unsold_tickets_stats";
```

Data Output Messages Notifications			
≡ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	Номер поезда integer	Тип вагона character varying	Количество bigint
1	102	Плацкарт	15
2	108	Сидячий	15
3	106	СВ	15
4	107	Плацкарт	15
5	105	Купе	15
6	109	Общий	15
7	103	Сидячий	15
8	110	Купе	15
9	104	Общий	15
10	101	СВ	15

5.3. Создать запросы на модификацию данных

- INSERT

	ID_билета [PK] integer	ID_типа билета integer	ID_паспорта integer	ID_кассы integer	Дата прибытия date	Точка посадки character varying	Точка высадки character varying	Способ оплаты character varying	Статус character varying
1	501	2	1	2	2026-04-21	[null]	[null]	[null]	Оплачен
2	502	3	2	3	2026-04-21	[null]	[null]	[null]	Оплачен
3	503	4	3	4	2026-04-21	[null]	[null]	[null]	Оплачен
4	504	5	4	5	2026-04-21	[null]	[null]	[null]	Оплачен
5	505	1	5	1	2026-04-21	[null]	[null]	[null]	Оплачен
6	506	2	6	2	2026-04-21	[null]	[null]	[null]	Оплачен
7	507	3	7	3	2026-04-21	[null]	[null]	[null]	Оплачен
8	508	4	8	4	2026-04-21	[null]	[null]	[null]	Оплачен
9	509	5	9	5	2026-04-21	[null]	[null]	[null]	Оплачен
10	510	1	10	1	2026-04-21	[null]	[null]	[null]	Оплачен
11	511	2	11	2	2026-04-23	[null]	[null]	[null]	Оплачен
12	512	3	12	3	2026-04-23	[null]	[null]	[null]	Оплачен
13	513	4	13	4	2026-04-23	[null]	[null]	[null]	Оплачен
14	514	5	14	5	2026-04-23	[null]	[null]	[null]	Оплачен
15	515	1	15	1	2026-04-23	[null]	[null]	[null]	Оплачен
16	516	2	16	2	2026-04-23	[null]	[null]	[null]	Оплачен
17	517	3	17	3	2026-04-23	[null]	[null]	[null]	Оплачен
18	518	4	18	4	2026-04-23	[null]	[null]	[null]	Оплачен
19	519	5	19	5	2026-04-23	[null]	[null]	[null]	Оплачен
20	520	1	20	1	2026-04-23	[null]	[null]	[null]	Оплачен
21	521	2	21	2	2026-04-23	[null]	[null]	[null]	Оплачен
22	522	3	22	3	2026-04-23	[null]	[null]	[null]	Оплачен
23	523	4	23	4	2026-04-23	[null]	[null]	[null]	Оплачен
24	524	5	24	5	2026-04-23	[null]	[null]	[null]	Оплачен
25	525	1	25	1	2026-04-23	[null]	[null]	[null]	Оплачен
26	526	2	26	2	2026-04-23	[null]	[null]	[null]	Оплачен
27	527	3	27	3	2026-04-23	[null]	[null]	[null]	Оплачен
28	528	4	28	4	2026-04-23	[null]	[null]	[null]	Оплачен
29	529	5	29	5	2026-04-23	[null]	[null]	[null]	Оплачен
30	530	1	30	1	2026-04-23	[null]	[null]	[null]	Оплачен

```

INSERT INTO "Passengers"."Билет" ("ID_билета", "ID_типа билета", "ID_паспорта", "ID_кассы", "Дата прибытия", "Статус")
VALUES (
    (SELECT MAX("ID_билета") + 1 FROM "Passengers"."Билет"), -- Генерация нового ID
    1, -- Тип билета 'Полный'
    (SELECT "ID_паспорта" FROM "Passengers"."Паспорт" p
     JOIN "Passengers"."Пассажир" pass ON p."ID_пассажира" = pass."ID_пассажира"
     WHERE pass."Фамилия" = 'Иванов' LIMIT 1),
    (SELECT "ID_кассы" FROM "Passengers"."Касса" WHERE "Город" = 'Москва' LIMIT 1),
    CURRENT_DATE + 1,
    'Оформлен'
);

```

- (SELECT MAX("ID_билета") + 1 FROM "Passengers"."Билет"): Генерация нового ID
- (SELECT "ID_паспорта" FROM "Passengers"."Паспорт" p JOIN "Passengers"."Пассажир" pass ... WHERE pass."Фамилия" = 'Иванов' LIMIT 1): Поиск пассажира по фамилии
- **LIMIT 1** страхует от ошибок, если в базе несколько Ивановых.
- (SELECT "ID_кассы" FROM "Passengers"."Касса" WHERE "Город" = 'Москва' LIMIT 1): Поиск кассы по городу
- CURRENT_DATE + 1: Работа с датой и статусом

Data Output Messages Notifications										
	ID_билета [PK] integer	ID_типа билета integer	ID_паспорта integer	ID_кассы integer	Дата прибытия date	Точка посадки character varying	Точка высадки character varying	Способ оплаты character varying	Статус character varying	
11	511	2	11	2	2026-04-23	[null]	[null]	[null]	Оплачен	
12	512	3	12	3	2026-04-23	[null]	[null]	[null]	Оплачен	
13	513	4	13	4	2026-04-23	[null]	[null]	[null]	Оплачен	
14	514	5	14	5	2026-04-23	[null]	[null]	[null]	Оплачен	
15	515	1	15	1	2026-04-23	[null]	[null]	[null]	Оплачен	
16	516	2	16	2	2026-04-23	[null]	[null]	[null]	Оплачен	
17	517	3	17	3	2026-04-23	[null]	[null]	[null]	Оплачен	
18	518	4	18	4	2026-04-23	[null]	[null]	[null]	Оплачен	
19	519	5	19	5	2026-04-23	[null]	[null]	[null]	Оплачен	
20	520	1	20	1	2026-04-23	[null]	[null]	[null]	Оплачен	
21	521	2	21	2	2026-04-23	[null]	[null]	[null]	Оплачен	
22	522	3	22	3	2026-04-23	[null]	[null]	[null]	Оплачен	
23	523	4	23	4	2026-04-23	[null]	[null]	[null]	Оплачен	
24	524	5	24	5	2026-04-23	[null]	[null]	[null]	Оплачен	
25	525	1	25	1	2026-04-23	[null]	[null]	[null]	Оплачен	
26	526	2	26	2	2026-04-23	[null]	[null]	[null]	Оплачен	
27	527	3	27	3	2026-04-23	[null]	[null]	[null]	Оплачен	
28	528	4	28	4	2026-04-23	[null]	[null]	[null]	Оплачен	
29	529	5	29	5	2026-04-23	[null]	[null]	[null]	Оплачен	
30	530	1	30	1	2026-04-23	[null]	[null]	[null]	Оплачен	
31	531	1	1	1	2026-04-23	[null]	[null]	[null]	Оформлен	

● UPDATE

	ID_типа билета [PK] integer	Название character varying	Базовая цена integer	Правила скидок character varying
1	1	Полный	3500	Нет
2	2	Детский	1750	50%
3	3	Студенческий	2600	25%
4	4	Льготный	1200	66%
5	5	Люкс	9500	Нет
6	6	Бизнес	15000	Нет
7	7	Групповой	2500	От 10 чел
8	8	Военный	1800	Спец
9	9	Пенсионный	1750	50%
10	10	Промо	900	Акция


```

UPDATE "Passengers"."Тип билета"
SET "Базовая цена" = "Базовая цена" * 1.1
WHERE "ID_типа билета" IN (
    SELECT DISTINCT b."ID_типа билета"
    FROM "Passengers"."Билет" b
    JOIN "Passengers"."Касса" k ON b."ID_кассы" = k."ID_кассы"
    WHERE k."Статус" = 'Да'
);

```

- SET "Базовая цена" = "Базовая цена" * 1.1: Умножение на **1.1** эквивалентно прибавлению 10% к текущей стоимости.
- SELECT DISTINCT b."ID_типа билета": Выбирает уникальные идентификаторы типов билетов.
- JOIN "Passengers"."Касса" k ON ...: Связывает таблицу билетов с кассами.
- WHERE k."Статус" = 'Да': Ключевое условие. Подзапрос находит только те типы билетов, которые продаются в кассах с активным статусом («Да»).

Data Output Messages Notifications				
	ID_типа билета [PK] integer	Название character varying	Базовая цена integer	Правила скидок character varying
1	1	Полный	3850	Нет
2	2	Детский	1925	50%
3	3	Студенческий	2860	25%
4	4	Льготный	1320	66%
5	5	Люкс	10450	Нет
6	6	Бизнес	15000	Нет
7	7	Групповой	2500	От 10 чел
8	8	Военный	1800	Спец
9	9	Пенсионный	1750	50%
10	10	Промо	900	Акция

- DELETE

	ID_места [PK] integer 	ID_билета integer 	ID_типа места integer 	Номер места integer 	Статус character varying 
5	5	505	1	5	Занято
6	6	506	2	6	Занято
7	7	507	3	7	Занято
8	8	508	4	8	Занято
9	9	509	5	9	Занято
10	10	510	1	10	Занято
11	11	511	2	11	Занято
12	12	512	3	12	Занято
13	13	513	4	13	Занято
14	14	514	5	14	Занято
15	15	515	1	15	Занято
16	16	[null]	2	16	Свободно
17	17	[null]	3	17	Свободно
18	18	[null]	4	18	Свободно
19	19	[null]	5	19	Свободно
20	20	[null]	1	20	Свободно
21	21	[null]	2	21	Свободно
22	22	[null]	3	22	Свободно
23	23	[null]	4	23	Свободно
24	24	[null]	5	24	Свободно
25	25	[null]	1	25	Свободно
26	26	[null]	2	26	Свободно
27	27	[null]	3	27	Свободно
28	28	[null]	4	28	Свободно
29	29	[null]	5	29	Свободно
30	30	[null]	1	30	Свободно
31	200	[null]	1	10	Свободно
32	201	[null]	2	11	Свободно
33	202	[null]	1	12	Свободно
34	999	[null]	1	12	Свободно

```
DELETE FROM "Passengers"."Место"
WHERE "ID_билета" IN (
    SELECT "ID_билета"
    FROM "Passengers"."Билет"
    WHERE "Статус" = 'Оплачен'
);
```

- SELECT "ID_билета" FROM "Passengers"."Билет" WHERE "Статус" = 'Оплачен': Этот вложенный запрос обращается к таблице билетов и находит все ID тех билетов, которые имеют официальный статус «Оплачен»

	ID_места [PK] integer	ID_билета integer	ID_типа места integer	Номер места integer	Статус character varying
1	16	[null]	2	16	Свободно
2	17	[null]	3	17	Свободно
3	18	[null]	4	18	Свободно
4	19	[null]	5	19	Свободно
5	20	[null]	1	20	Свободно
6	21	[null]	2	21	Свободно
7	22	[null]	3	22	Свободно
8	23	[null]	4	23	Свободно
9	24	[null]	5	24	Свободно
10	25	[null]	1	25	Свободно
11	26	[null]	2	26	Свободно
12	27	[null]	3	27	Свободно
13	28	[null]	4	28	Свободно
14	29	[null]	5	29	Свободно
15	30	[null]	1	30	Свободно
16	200	[null]	1	10	Свободно
17	201	[null]	2	11	Свободно
18	202	[null]	1	12	Свободно
19	999	[null]	1	12	Свободно

5.4. Создать индексы

Создать планы

```
EXPLAIN ANALYZE
SELECT * FROM "Passengers"."Касса"
WHERE "Город" = 'Москва' AND "Статус" = 'Да';
```

На вкладке **Data Output** появится подробный отчет, в конце которого будут строки **Planning Time** и **Execution Time**. Именно эти цифры нужно сравнивать для отчета при работе с индексами.

- Без индекса

Data Output Messages Notifications	
	QUERY PLAN text
1	Seq Scan on "Касса" (cost=0.00..28.00 rows=266 width=64) (actual time=0.016..0.162 rows=267.00 loop...
2	Filter: (((("Город")::text = 'Москва'::text) AND (("Статус")::text = 'Да'::text)))
3	Rows Removed by Filter: 733
4	Buffers: shared hit=13
5	Planning Time: 0.085 ms
6	Execution Time: 0.185 ms

В плане будет написано **Seq Scan on "Касса"**

- С индексами
- ❖ Создаем индекс, который оптимизирует поиск сразу по двум полям.

```
CREATE INDEX idx_kassa_city_status
ON "Passengers"."Касса" ("Город", "Статус");
```

- ❖ Повторим запрос

QUERY PLAN	
text	
1	Bitmap Heap Scan on "Касса" (cost=6.88..23.87 rows=266 width=64) (actual time=0.041..0.076 rows=267.00 loops=1)
2	Recheck Cond: (((("Город")::text = 'Москва'::text) AND (("Статус")::text = 'Да'::text))
3	Heap Blocks: exact=12
4	Buffers: shared hit=13
5	-> Bitmap Index Scan on idx_kassa_city_status (cost=0.00..6.81 rows=266 width=0) (actual time=0.028..0.028 rows=267.00 loo...
6	Index Cond: (((("Город")::text = 'Москва'::text) AND (("Статус")::text = 'Да'::text))
7	Index Searches: 1
8	Buffers: shared hit=1
9	Planning Time: 0.115 ms
10	Execution Time: 0.103 ms

Планировщик должен переключиться **Bitmap Index Scan**

- Удалить индексы

```
DROP INDEX "Passengers".idx_kassa_city_status;
```

Обычно команда **DROP INDEX** блокирует таблицу для записи на время выполнения. Если база данных находится в активном использовании, можно использовать **DROP INDEX CONCURRENTLY**, чтобы удаление произошло в фоновом режиме без блокировки таблицы.

6. Вывод

В ходе выполнения лабораторной работы были закреплены практические навыки проектирования и реализации сложных SQL-запросов в среде pgAdmin 4 для СУБД PostgreSQL.

Основные результаты работы:

- Сложные выборки данных: Были успешно реализованы запросы на выборку, включающие многотабличные объединения (JOIN), использование агрегатных функций (SUM, COUNT) и условий группировки (GROUP BY, HAVING). Это позволило получить аналитическую информацию о выручке, заполняемости поездов и наличии свободных мест в реальном времени.
- Модификация данных с подзапросами: На практике реализованы операции вставки (INSERT), обновления (UPDATE) и удаления (DELETE) записей. Использование вложенных подзапросов позволило автоматизировать процесс изменения данных, обеспечивая ссылочную целостность без необходимости ручного ввода идентификаторов.

- Использование представлений (Views): Были созданы виртуальные таблицы для часто используемых отчетов (статистика свободных мест). Это продемонстрировало возможности абстракции данных, упрощения доступа к сложным структурам и повышения удобства работы конечного пользователя.
- Оптимизация производительности: С помощью команды EXPLAIN ANALYZE был проведен анализ планов выполнения запросов. Экспериментальным путем доказано влияние индексов на скорость поиска данных: на больших объемах данных использование индексов переводит процесс из последовательного сканирования (Seq Scan) в индексное (Index Scan), что значительно сокращает время выполнения (Execution Time).

Работа подтвердила эффективность реляционной модели данных и языка SQL для управления сложными информационными системами. Полученные навыки позволяют не только извлекать информацию, но и эффективно управлять производительностью и структурой базы данных.